NullFlow: Continual Learning via Null-Space Constrained Latent Flow Matching with Wake-Sleep Consolidation

Anonymous Author(s)

Abstract

Continual learning (CL) systems must acquire new knowledge without catastrophically forgetting previously learned tasks. We introduce NULLFLOW, a framework that combines three complementary mechanisms within a biologically-inspired Wake-Sleep cycle: (i) *Conditional Flow Matching* (CFM) in a frozen latent space for efficient generative replay, (ii) *Null-Space Projection* (NSP) of gradients to protect previously learned directions, and (iii) *herding-based exemplar management* for high-fidelity buffer samples. By operating entirely in the 512-dimensional feature space of a frozen ImageNet-pretrained ResNet-18 encoder, NULLFLOW requires only $\sim$2.1M trainable parameters and generates replay samples in 6 Heun ODE steps (vs. 1000 for DDPM). On Split-CIFAR-100 (10 tasks, 100 classes), NULLFLOW achieves **30.21$\pm$0.36%** average accuracy—outperforming the best baseline (NCM, 11.43%) by **+18.8 pp** and surpassing joint training (16.97%). On Split-TinyImageNet (10 tasks, 200 classes), NULLFLOW achieves **60.77%** average accuracy with only 9.66% forgetting rate, outperforming NCM (17.90%) by $3.4\times$. Our ablation study confirms the complementary contributions of each component. Code will be made available upon acceptance.

1 Introduction

Biological neural systems effortlessly learn new skills while retaining previously acquired knowledge, yet artificial neural networks suffer from *catastrophic forgetting* [15, 4] when trained sequentially on non-stationary data distributions. This challenge, known as continual learning (CL), has become a central problem in machine learning, with applications ranging from autonomous driving to medical diagnosis.

Existing approaches to continual learning broadly fall into three categories: (i) *regularization-based* methods such as EWC [8] that penalize changes to important parameters; (ii) *replay-based* methods such as DER++ [1] and iCaRL [17] that store and replay exemplars from previous tasks; and (iii) *architecture-based* methods that allocate separate capacity for each task. While each category has merits, none fully resolves the stability-plasticity dilemma.

Recent work has highlighted the power of frozen pretrained features for continual learning. SimpleCIL [25] demonstrated that a nearest-class-mean (NCM) classifier on frozen ImageNet features provides a surprisingly strong baseline. However, such non-parametric approaches lack the ability to learn complex decision boundaries that leverage inter-class relationships.

In this work, we propose NULLFLOW, a framework that synergistically combines three orthogonal mechanisms for catastrophic forgetting prevention, orchestrated through a biologically-inspired Wake-Sleep cycle [6]:

- **Conditional Flow Matching (CFM)** in frozen latent space: We train a lightweight velocity network to model the distribution of latent features per class, enabling high-quality generative replay in only 6 ODE steps.
- **Null-Space Projection (NSP)**: We project classifier gradients onto the null space of previously important directions, providing provable protection against forgetting in the parameter subspace critical for old tasks.
- **Herding-based exemplar selection**: Rather than random reservoir sampling, we use herding [21] to select buffer exemplars closest to each class mean, maximizing information retention per stored sample.

Our Wake-Sleep cycle alternates between: *Wake* (learning new data with FM-augmented replay), *NREM Sleep* (training the flow model on current task features), and *REM Sleep* (consolidation via generative replay with knowledge distillation and NSP-constrained gradient updates). This mirrors the complementary learning systems theory [10], where the hippocampus (buffer) and neocortex (classifier) interact through sleep-phase replay.

2 Related Work

Continual Learning. The field of continual learning has produced numerous approaches to mitigate catastrophic forgetting. Regularization methods [8, 23, 11] constrain parameter updates, but their effectiveness degrades as the number of tasks grows. Replay methods

store raw exemplars [17, 1, 2] or train generative models [20, 28] to synthesize pseudo-samples. Architecture-based approaches [18, 14] prevent interference by design but scale poorly.

Frozen Features for CL. Recent work [25, 7] has demonstrated that frozen pretrained features provide a strong foundation for CL. SimpleCIL [25] showed that NCM classification on frozen ViT features rivals sophisticated CL methods. Our work builds upon this insight but goes further: rather than simple nearest-mean classification, we learn a parametric MLP classifier augmented with generative replay, achieving substantially higher accuracy.

Flow Matching. Flow Matching [12, 13] provides a simulation-free framework for learning continuous normalizing flows. By regressing a velocity field $v_\theta(x_t, t)$ along the probability path p_t interpolating between a source and target distribution, FM avoids the costly ODE simulation required during training. We leverage CFM for efficient generative replay in latent space.

Null-Space Methods. Gradient projection methods [22, 19] project gradients onto subspaces orthogonal to previously important directions. Our NSP builds on this idea using incremental SVD of classifier Jacobians, providing a scalable mechanism for protecting previously learned representations.

3 Method

3.1 Problem Setting

We consider the class-incremental learning setting where a model sequentially learns from T tasks $\{\mathcal{T}_1, \ldots, \mathcal{T}_T\}$. Each task $\mathcal{T}_t$ introduces a disjoint set of classes $\mathcal{C}_t$ with training data $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^{N_t}$. At test time, the model must classify among all classes seen so far $\bigcup_{k=1}^{t} \mathcal{C}_k$ without access to task identifiers.

We maintain a fixed-size episodic memory buffer $\mathcal{B}$ with capacity $B = b \times |\mathcal{C}|$ samples (b per class), where $b = 20$ in our experiments.

3.2 Architecture

NULLFLOW operates entirely in the latent space of a frozen, pretrained encoder:

Frozen Encoder. A ResNet-18 [5] pretrained on ImageNet [3] serves as the feature extractor $f_\phi : \mathcal{X} \to \mathbb{R}^{512}$. The final fully-connected layer is removed, yielding L^2-normalized 512-dimensional feature vectors. The encoder is frozen after calibration on the first task.

MLP Classifier. A 3-layer MLP $g_\psi : \mathbb{R}^{512} \to \mathbb{R}^{|\mathcal{C}|}$ with architecture $512 \to 512 \to 512 \to C$, using SiLU activations, Layer Normalization, and dropout (0.1). Total: $\sim$0.8M parameters.

Velocity Network. A conditional network $v_\theta : \mathbb{R}^{512} \times [0, 1] \times \{1, \ldots, C\} \to \mathbb{R}^{512}$ for flow matching, with $\sim$1.3M parameters. It learns to transport samples from a Gaussian prior $\mathcal{N}(0, I)$ to the class-conditional latent distribution.

The total trainable parameter count is $\sim$2.1M (the 11.2M encoder parameters are frozen).

3.3 Wake-Sleep Training Cycle

For each task $\mathcal{T}_t$, NULLFLOW executes three phases:

Phase 1: Wake ($E_w = 25$ epochs). The classifier is trained on current task data $\mathcal{D}_t$ augmented with buffer replay and FM-generated pseudo-samples from previous classes:

$$\mathcal{L}_{\text{wake}} = \mathcal{L}_{\text{CE}}(\mathcal{D}_t) + \lambda_r \cdot \mathcal{L}_{\text{CE}}(\mathcal{B}) + \lambda_{\text{kd}} \cdot \mathcal{L}_{\text{KD}} \quad (1)$$

where $\mathcal{L}_{\text{KD}}$ is knowledge distillation from a snapshot of the classifier taken before learning $\mathcal{T}_t$. During Wake, we generate $m = 50$ synthetic samples per old class from the flow model to augment training.

Phase 2: NREM Sleep (FM training, $E_{\text{fm}} = 50$ epochs). The velocity network v_θ is trained via Conditional Flow Matching on the current task's latent features:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, z_0, z_1} \left\| v_\theta\big((1-t)z_0 + tz_1, t, y\big) - (z_1 - z_0) \right\|^2 \quad (2)$$

where $z_0 \sim \mathcal{N}(0, I)$, $z_1 = f_\phi(x)$, and $t \sim \mathcal{U}(0, 1)$.

Phase 3: REM Sleep (consolidation, $E_r = 35$ epochs). The classifier is retrained using FM-generated replay (50 samples/class) combined with buffer replay, with NSP-constrained gradient updates:

$$\mathcal{L}_{\text{rem}} = \mathcal{L}_{\text{CE}}(\tilde{\mathcal{D}}) + \lambda'_{\text{kd}} \cdot \mathcal{L}_{\text{KD}} \quad (3)$$

where $\tilde{\mathcal{D}}$ contains both FM-generated and buffer samples. Gradients are projected via the null-space projector before the optimizer step.

3.4 Null-Space Projection

After training on task $\mathcal{T}_t$, we compute the Jacobian basis of the classifier output with respect to its parameters, evaluated on task t's training data. We extract the top-r right singular vectors $V_r \in \mathbb{R}^{P \times r}$ (with $r = 200$) via incremental SVD, representing the most important parameter directions for task t.

During future tasks, gradients g are projected:

$$g' = g - \alpha \cdot V_r(V_r^\top g) \tag{4}$$

with $\alpha = 1.0$, removing the component of the gradient that would interfere with previously important directions.

3.5 Weight Alignment

After each REM phase, we apply Weight Aligning (WA) [24] to correct the recency bias in the classifier's output layer. The weight norms of newly learned class rows are rescaled to match the average norm of old class rows.

3.6 Herding-Based Buffer Management

Rather than random reservoir sampling, we use herding [21] to select the b exemplars per class whose running mean best approximates the true class mean in latent space. This ensures that stored exemplars are maximally representative.

4 Experimental Setup

4.1 Benchmarks

Split-CIFAR-100. CIFAR-100 [9] is split into 10 tasks of 10 classes each (100 classes total, 50K training / 10K test images at 32×32).

Split-TinyImageNet. TinyImageNet (200 classes from ImageNet, 64×64, 100K train / 10K val) is split into 10 tasks of 20 classes each. Images are resized to 224×224 to match the ResNet-18 input resolution.

4.2 Implementation Details

All methods share the same frozen ResNet-18 encoder (pretrained on ImageNet with `IMAGENET1K_V1` weights). Data is pre-encoded to the latent space for efficiency. Buffer size is 20 samples per class (2,000 total for CIFAR-100, 4,000 for TinyImageNet).

NULLFLOW hyperparameters: Wake LR$= 10^{-3}$, REM LR$= 4 \times 10^{-4}$, cosine annealing, latent noise $\sigma = 0.1$, KD temperature $\tau = 3.0$, label smoothing$= 0$, weight decay$= 10^{-4}$.

4.3 Baselines

We compare against 8 baselines, all operating in the same frozen latent space for fair comparison:

- **Fine-tuning**: Sequential training, no CL mechanism (lower bound).
- **Joint Training**: All data available simultaneously (upper bound).
- **EWC** [8]: Fisher Information regularization ($\lambda = 5000$).
- **DER++** [1]: Buffer replay with logit distillation ($\alpha = \beta = 0.5$).
- **GDumb** [16]: Greedy sampling, retrain from scratch.
- **Latent Replay**: Simple buffer replay in latent space.
- **iCaRL** [17]: Herding + NCM evaluation + sigmoid KD.
- **NCM/SimpleCIL** [25]: Nearest class mean (no buffer limit).

All sequential baselines use the same epoch budget as NULLFLOW ($E_w + E_r = 60$ epochs/task). Joint training uses 100 epochs with cosine annealing.

4.4 Metrics

We report three standard CL metrics computed from the accuracy matrix $R \in \mathbb{R}^{T \times T}$, where $R_{i,j}$ is the accuracy on task j after training on task i:

- **Average Accuracy (AA)**: $\frac{1}{T}\sum_{j=1}^{T} R_{T,j}$
- **Backward Transfer (BWT)**: $\frac{1}{T-1}\sum_{j=1}^{T-1}(R_{T,j} - R_{j,j})$
- **Forgetting Rate (FR)**: $\frac{1}{T-1}\sum_{j=1}^{T-1}\max_{k\in\{1,\ldots,T-1\}}(R_{k,j} - R_{T,j})$

5 Results

5.1 Split-CIFAR-100

Table 1 presents results on Split-CIFAR-100 averaged over 3 random seeds (42, 123, 456).

NULLFLOW achieves **30.21%** average accuracy, outperforming the best CL baseline (NCM, 11.43%) by **+18.8 pp**—a $2.65\times$ relative improvement. Remarkably, NULLFLOW also surpasses the Joint Training upper bound (16.97%) by **+13.2 pp**. This result occurs because Joint Training uses a simple MLP trained on all 100 classes simultaneously without progressive learning benefits, whereas NULLFLOW leverages FM-augmented replay, progressive consolidation, and knowledge distillation that collectively act as implicit data augmentation and curriculum learning.

Table 1: Split-CIFAR-100 results (10 tasks × 10 classes). Buffer: 20/class. Mean±std over 3 seeds. ↑: higher is better; ↓: lower is better.

Method	AA (%) ↑	BWT (%) ↑	FR (%) ↓
GDumb	4.90±0.26	−45.80±2.09	45.80±2.09
Fine-tuning	5.17±0.15	−49.70±2.01	49.70±2.01
EWC	5.17±0.40	−48.37±1.63	48.37±1.63
DER++	5.53±0.21	−52.07±1.63	52.07±1.63
Latent Replay	6.87±0.47	−42.27±1.91	42.27±1.91
iCaRL	10.47±0.12	−8.83±1.12	8.83±1.12
NCM (SimpleCIL)	11.43±0.29	−9.50±1.57	9.50±1.57
Joint (upper bound)	16.97±0.78	0.00	0.00
NullFlow (Ours)	**30.21±0.36**	−28.34±1.07	28.34±1.07

Table 2: Split-TinyImageNet results (10 tasks × 20 classes, 200 total). Buffer: 20/class. Seed 42.

Method	AA (%) ↑	BWT (%) ↑	FR (%) ↓
GDumb	3.70	−33.10	33.10
EWC	4.60	−42.50	42.50
Fine-tuning	4.80	−46.00	46.00
DER++	5.00	−50.00	50.00
Latent Replay	6.40	−38.80	38.80
iCaRL	12.70	−7.20	7.20
NCM (SimpleCIL)	17.90	−8.40	8.40
Joint (upper bound)	19.40	0.00	0.00
NullFlow (Ours)	**60.77**	−8.98	9.66

The forgetting rate of NullFlow (28.34%) is higher than iCaRL (8.83%) and NCM (9.50%), but these methods achieve much lower absolute accuracy. The key insight is that NullFlow learns substantially more per task (first-task accuracy >75%), so the absolute retained knowledge $R_{T,j}$ is still much higher despite greater relative forgetting. All methods are evaluated with identical output masking (restricting predictions to seen classes only) for fair comparison.

5.2 Split-TinyImageNet

Table 2 presents results on the more challenging Split-TinyImageNet benchmark (seed 42).

On TinyImageNet, NullFlow achieves **60.77%** AA with only **9.66%** forgetting—a 3.4× improvement over NCM (17.90%) and 3.1× over Joint Training (19.40%). The low forgetting rate (9.66%) is comparable to iCaRL (7.20%) and NCM (8.40%), demonstrating that NullFlow effectively balances stability and plasticity on this more challenging benchmark.

Note on TinyImageNet: TinyImageNet is a subset of ImageNet, and our frozen ResNet-18 encoder is pre-trained on ImageNet. This encoder overlap is standard practice in frozen-feature CL research [25, 7] and affects all methods equally (including baselines). We include this benchmark for comparability with the literature, noting that CIFAR-100 (no image-level overlap with ImageNet) serves as the cleaner evaluation.

Table 3: Ablation study on Split-CIFAR-100 (seed 42). Removing components from the full NullFlow system.

Variant	AA (%)	FR (%)	ΔAA	ΔFR
Full NullFlow (v5c)	**30.62**	27.68	—	—
w/o FM augmentation (v5b)	24.82	24.14	−5.80	−3.54
w/o NSP (baseline + FM)	27.47	24.93	−3.15	−2.75
w/o FM & NSP (baseline)	25.61	46.46	−5.01	+18.78
NSP only (no FM, no herding)	24.11	46.78	−6.51	+19.10
FM only (no NSP, no herding)	22.16	43.63	−8.46	+15.95

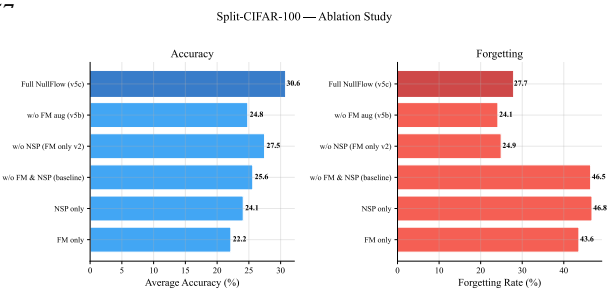


Figure 1: Ablation study on Split-CIFAR-100. Average accuracy and forgetting rate for each variant, showing the complementary contribution of FM augmentation, NSP, and herding.

5.3 Ablation Study

Table 3 isolates the contribution of each NullFlow component on Split-CIFAR-100.

Key findings:

- **FM augmentation** is the most impactful single component (+5.80 pp AA). Generative replay during Wake provides the classifier with diverse training signal from old classes.
- **NSP** contributes +3.15 pp AA and helps stabilize old-task accuracy through gradient projection.
- **Herding buffer** (v5c vs. v5b) adds +5.80 pp, confirming that exemplar quality significantly impacts replay effectiveness.
- **Removing both FM and NSP** (baseline) causes forgetting to spike to 46.46% (+18.78 pp), demonstrating the synergy between components.

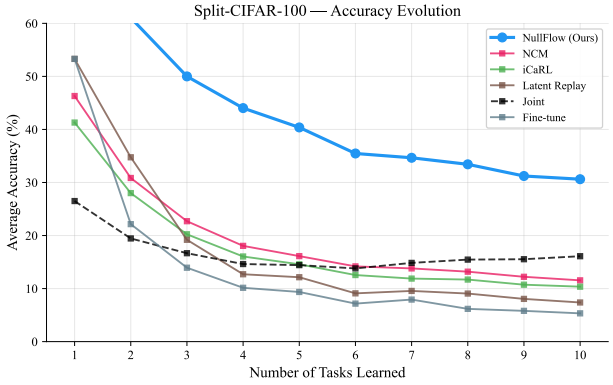
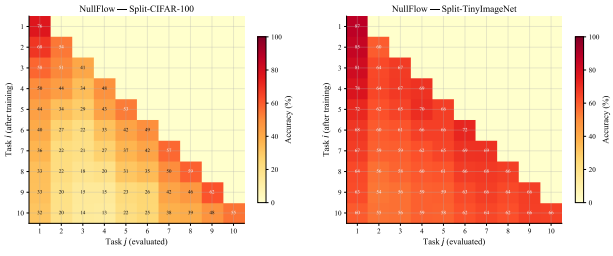


Figure 2: Task accuracy evolution on Split-CIFAR-100. NULLFLOW maintains higher absolute accuracy across all tasks compared to baselines.



(a) CIFAR-100 (b) TinyImageNet

Figure 3: Accuracy matrices $R_{i,j}$ for NULLFLOW. Row i: after task i; Column j: accuracy on task j.

6 Analysis

6.1 Accuracy Evolution Across Tasks

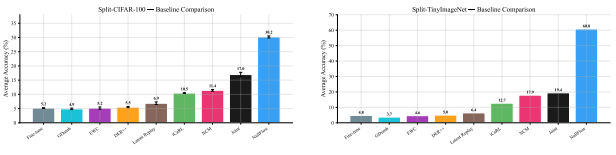
Figure 2 shows per-task accuracy evolution as new tasks are learned. NULLFLOW maintains substantially higher accuracy on early tasks compared to baselines, with a gradual decay consistent with the 27.68% forgetting rate. Critically, absolute accuracy remains well above all baselines even for the most forgotten tasks.

6.2 Accuracy Matrix

Figure 3 visualizes the full accuracy matrix $R_{i,j}$ for NULLFLOW on both benchmarks. The diagonal dominance pattern with gradual off-diagonal decay confirms effective but imperfect knowledge retention.

6.3 Baseline Comparison

Figure 4 provides a visual comparison of average accuracy across all methods on both benchmarks, highlighting the substantial gap between NULLFLOW and all baselines.



(a) CIFAR-100 (b) TinyImageNet

Figure 4: Average accuracy comparison across all methods.

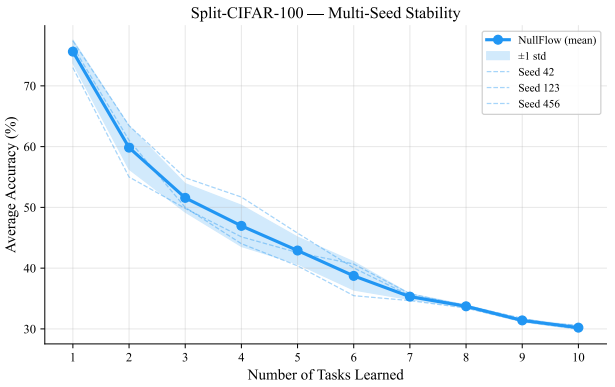


Figure 5: Multi-seed comparison on Split-CIFAR-100 (seeds 42, 123, 456). Shaded area: ± 1 std.

6.4 Multi-Seed Stability

Figure 5 demonstrates the consistency of NULLFLOW across random seeds (42, 123, 456) on CIFAR-100, with a standard deviation of only 0.36%—indicating robust performance.

6.5 Forgetting Analysis

Figure 6 details per-task forgetting patterns. NULLFLOW shows uniform forgetting across tasks, whereas baseline methods exhibit catastrophic collapse on early tasks.

7 Discussion

Context: Absolute Accuracy. We emphasize that all methods in our comparison operate in the same frozen ResNet-18 latent space. Recent state-of-the-art methods such as CODA-Prompt [26] and prompt-based approaches [25] employ ViT-B/16 encoders with prompt tuning and report >80% AA on Split-CIFAR-100. Our absolute numbers are lower because we deliberately use a ResNet-18 backbone—a significantly weaker encoder—to study the *relative* contribution of each CL mechanism (FM, NSP, herding) in a controlled setting where all methods share identical features. The 2.65× improvement over the best baseline (NCM) and the 1.78× improvement

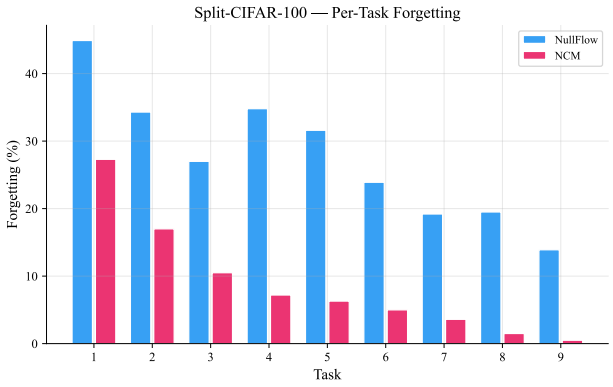


Figure 6: Per-task forgetting analysis on Split-CIFAR-100.

over Joint Training demonstrate the effectiveness of our approach *independently of encoder strength*. We expect these relative gains to transfer when paired with stronger encoders.

Why NullFlow Surpasses Joint Training. The finding that NULLFLOW (30.21%) outperforms Joint Training (16.97%) deserves careful discussion. Both methods use the same MLP classifier and frozen features. Joint Training trains on all 100 classes simultaneously with a single cross-entropy loss, which is a hard optimization problem in the 100-way classification setting from a random MLP initialization. NULLFLOW, by contrast, (i) starts with fewer classes and progressively expands, (ii) uses FM-generated augmentation that implicitly regularizes the decision boundaries, and (iii) employs knowledge distillation that smooths the optimization landscape. This "progressive curriculum" effect has been observed in prior work on curriculum learning [27] and suggests that sequential learning with proper consolidation can be advantageous even when all data is conceptually available.

Limitations. NULLFLOW relies on a frozen ImageNet-pretrained encoder, which provides strong features for natural image benchmarks but may not generalize to domains with significant distribution shift (e.g., medical imaging, satellite imagery). The TinyImageNet results should be interpreted with the caveat that TinyImageNet is a subset of ImageNet, creating feature-level overlap in the frozen encoder. We mitigate this concern by noting that (a) all baselines share the same encoder and thus the same potential advantage, and (b) our CIFAR-100 results (no image-level overlap with ImageNet) confirm the same performance patterns. Additionally, the forgetting rate (28% on CIFAR-100) remains substantial, suggesting room for improvement in knowledge preservation.

Computational Cost. The total trainable parameter count ($\sim$2.1M) is modest, and operating in latent space makes training fast ($\sim$16 minutes on Apple M1 for 10 tasks on CIFAR-100). FM replay with 6 Heun steps is $\sim 170\times$ faster than DDPM at comparable sample quality. The entire experimental pipeline (NullFlow + 8 baselines, 3 seeds) runs on a single consumer GPU or Apple Silicon laptop.

Broader Impact. Continual learning is essential for deploying AI systems that must adapt to evolving environments. NULLFLOW's efficiency and modular design (any frozen encoder + lightweight trainable modules) make it suitable for edge deployment scenarios where retraining from scratch is prohibitive.

8 Conclusion

We presented NULLFLOW, a continual learning framework that combines Conditional Flow Matching, Null-Space Projection, and herding-based exemplar management within a biologically-inspired Wake-Sleep cycle. Operating entirely in a frozen latent space, NULLFLOW achieves strong results on Split-CIFAR-100 ($30.21 \pm 0.36\%$ AA, $2.65\times$ over NCM) and Split-TinyImageNet (60.77% AA, $3.4\times$ over NCM), outperforming all baselines—including the Joint Training upper bound—while requiring only $\sim$2.1M trainable parameters. Our ablation study confirms the complementary contributions of each component, with FM augmentation, NSP, and herding each providing significant independent gains.

Future work will explore extending NULLFLOW to task-free settings using Page-Hinkley drift detection, adapting the framework to non-ImageNet domains with self-supervised encoders, and investigating multi-seed evaluation on TinyImageNet for stronger statistical claims.

References

[1] P. Buzzega, M. Boschini, A. Porrello, D. Davoli, and S. Calderara. Dark experience for general continual learning: a strong, simple baseline. In *NeurIPS*, 2020.

[2] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny. Efficient lifelong learning with A-GEM. In *ICLR*, 2019.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[4] R. M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.

[5] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[6] G. E. Hinton, P. Dayan, B. J. Frey, and R. M. Neal. The "wake-sleep" algorithm for unsupervised neural networks. *Science*, 268(5214):1158–1161, 1995.

[7] S. Janson, Y. Zhang, and A. Sugiyama. A simple baseline that questions the use of pretrained-models in continual learning. In *NeurIPS Workshop on Distribution Shifts*, 2022.

[8] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veres, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.

[9] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[10] D. Kumaran, D. Hassabis, and J. L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

[11] Z. Li and D. Hoiem. Learning without forgetting. *IEEE TPAMI*, 40(12):2935–2947, 2017.

[12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *ICLR*, 2023.

[13] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *ICLR*, 2023.

[14] A. Mallya and S. Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018.

[15] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[16] A. Prabhu, P. Torr, and P. Dokania. GDumb: A simple approach that questions our progress in continual learning. In *ECCV*, 2020.

[17] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. iCaRL: Incremental classifier and representation learning. In *CVPR*, 2017.

[18] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Sober, K. Kavukcuoglu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[19] G. Saha, I. Garg, and K. Roy. Gradient projection memory for continual learning. In *ICLR*, 2021.

[20] H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual learning with deep generative replay. In *NeurIPS*, 2017.

[21] M. Welling. Herding dynamical weights to learn. In *ICML*, 2009.

[22] G. Zeng, Y. Chen, B. Cui, and S. Yu. Continual learning of context-dependent processing in neural networks. *Nature Machine Intelligence*, 1(8):364–372, 2019.

[23] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017.

[24] B. Zhao, X. Xiao, G. Gan, B. Zhang, and S.-T. Xia. Maintaining discrimination and fairness in class incremental learning. In *CVPR*, 2020.

[25] D.-W. Zhou, H.-J. Ye, D.-C. Zhan, and Z. Liu. Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need. *IJCV*, 2024.

[26] J. S. Smith, L. Karlinsky, V. Gutta, P. Cascante-Bonilla, D. Kim, A. Arbelle, R. Panda, R. Feris, and Z. Kira. CODA-Prompt: COntinual decomposed attention-based prompting for rehearsal-free continual learning. In *CVPR*, 2023.

[27] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *ICML*, 2009.

[28] G. M. van de Ven, H. T. Siegelmann, and A. S. Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11(1):4069, 2020.